



ЗВІТ

Лабораторна робота № 6

Дисципліна: Базы даних та інформаційні системи

**Тема: Поглиблене вивчення MongoDB: оптимізація продуктивності,
використання шардінгу та реплікації**

Виконав

Студент 3 курсу

Групи МІТ-31

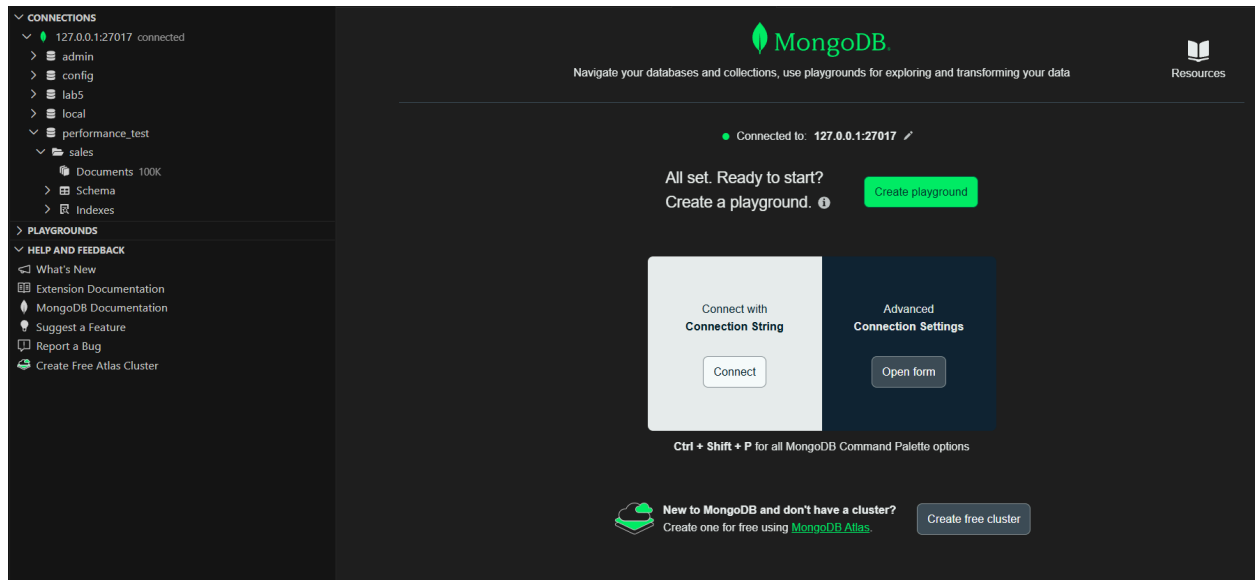
Факультету інформаційних технологій

Бабяк Володимир

Мета: Ознайомитися з методами підвищення продуктивності MongoDB. Навчитися використовувати індекси для оптимізації запитів. Дослідити механізми реплікації та шардінгу.

Хід роботи

Варіант 3: Доставка їжі



Підключення до локального сервера MongoDB через розширення MongoDB for VS Code та успішне створення бази даних performance_test із колекцією sales

```

1  from pymongo import MongoClient
2  import random
3  import datetime
4
5  # Підключення до локальної MongoDB
6  client = MongoClient("mongodb://localhost:27017")
7  db = client["performance_test"]
8  collection = db["sales"]
9
10 categories = ["Electronics", "Clothing", "Books", "Home", "Sports"]
11 documents = []
12
13 print("Генеруємо 100 000 записів. Це займе кілька секунд...")
14
15 # Генерація даних
16 for _ in range(100000):
17     documents.append({
18         "customer_id": random.randint(1, 1000),
19         "category": random.choice(categories),
20         "amount": round(random.uniform(5, 500), 2),
21         "timestamp": datetime.datetime(2024, random.randint(1, 12), random.randint(1, 28))
22     })
23
24 # Масова вставка в базу
25 collection.insert_many(documents)
26 print("100 000 документів успішно додано до колекції sales!")

```

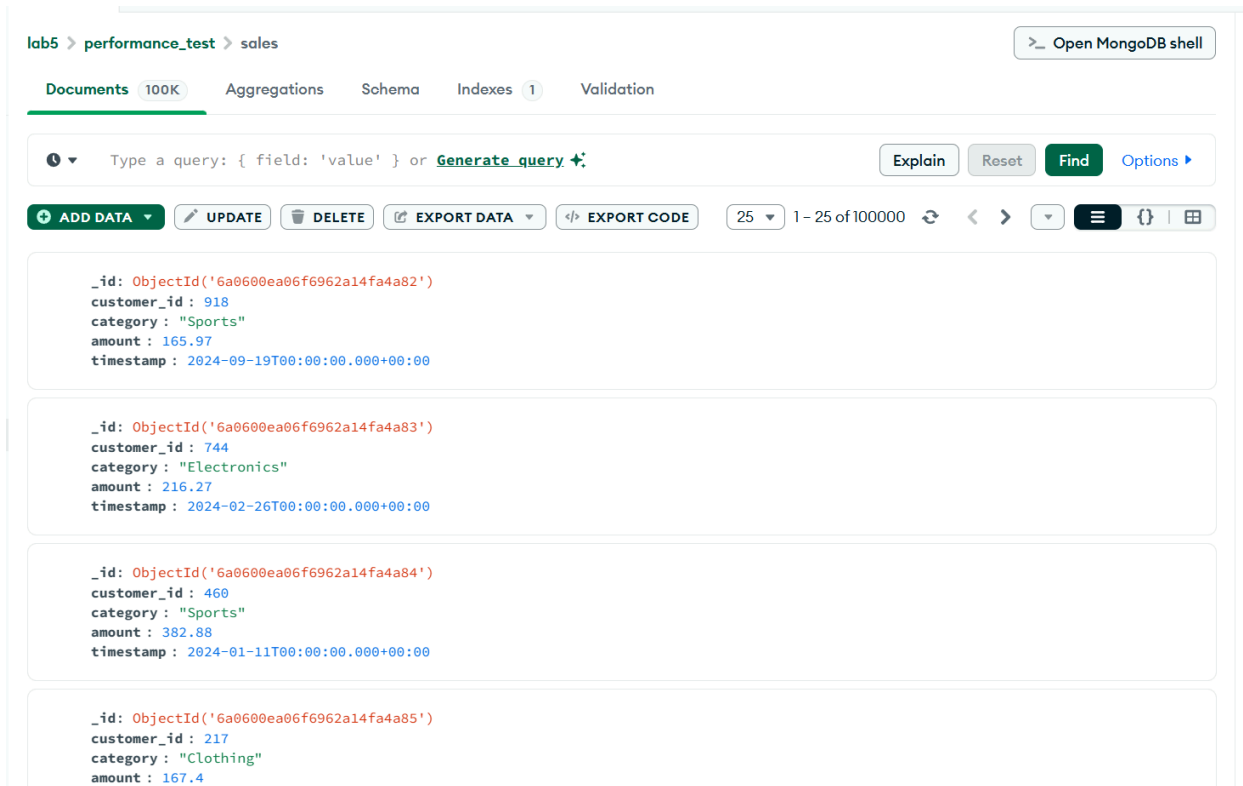
Програмний код розробленого скрипта

```

● PS C:\MongoDB\Labs> python insert_data.py
Генеруємо 100 000 записів. Це займе кілька секунд...
100 000 документів успішно додано до колекції sales!
○ PS C:\MongoDB\Labs>

```

Процес виконання скрипта в терміналі та підтвердження успішної генерації



Перевірка результату в інтерфейсі MongoDB Compass, де видно створену базу, колекцію та наявність у ній 100 000 документів

3. Виконання запиту без індексу та вимірювання часу виконання

Для перевірки продуктивності неоптимізованих запитів було створено скрипт `query_no_index.py`. Він підключається до нашої колекції `sales` та виконує пошук усіх документів, де поле `category` дорівнює `"Electronics"`. Щоб виміряти точний час виконання, використовувався модуль `time`. Запит повернув усі відповідні документи (їх кількість становить приблизно п'яту частину від 100 000, оскільки категорій п'ять), а час виконання склав близько 0.182045 секунд. База даних виконала повне сканування колекції (COLLSCAN), оскільки індексів ще немає.

```
query_no_index.py > ...
1  from pymongo import MongoClient
2  import time
3
4  # Підключення до бази даних
5  client = MongoClient("mongodb://localhost:27017")
6  db = client["performance_test"]
7  collection = db["sales"]
8
9  print("Виконуємо пошук без індексу...")
10
11 # Фіксуємо час початку
12 start_time = time.time()
13
14 # Виконуємо пошук документів, де категорія "Electronics", і перетворюємо в список
15 results = list(collection.find({"category": "Electronics"}))
16
17 # Фіксуємо час завершення
18 end_time = time.time()
19
```

Програмний код розробленого скрипта.

```
● PS C:\MongoDB\Labs> python query_no_index.py
Виконуємо пошук без індексу...
Знайдено документів: 20100
Time taken: 0.182045 seconds
○ PS C:\MongoDB\Labs> 
```

Процес виконання скрипта в терміналі та підтвердження успішної генерації

```
create_index.py > ...
1  from pymongo import MongoClient
2
3  # Підключення до бази даних MongoDB
4  client = MongoClient("mongodb://localhost:27017")
5
6  # Вибір бази даних і колекції
7  db = client["performance_test"]
8  collection = db["sales"]
9
10 # Створення індексу для поля "category" у колекції sales
11 collection.create_index("category")
12
13 # Виведення повідомлення
14 print("Індекс створено")
```

Terminal (Ctrl+`)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\MongoDB\Labs> python create_index.py
Індекс створено
PS C:\MongoDB\Labs> 
```

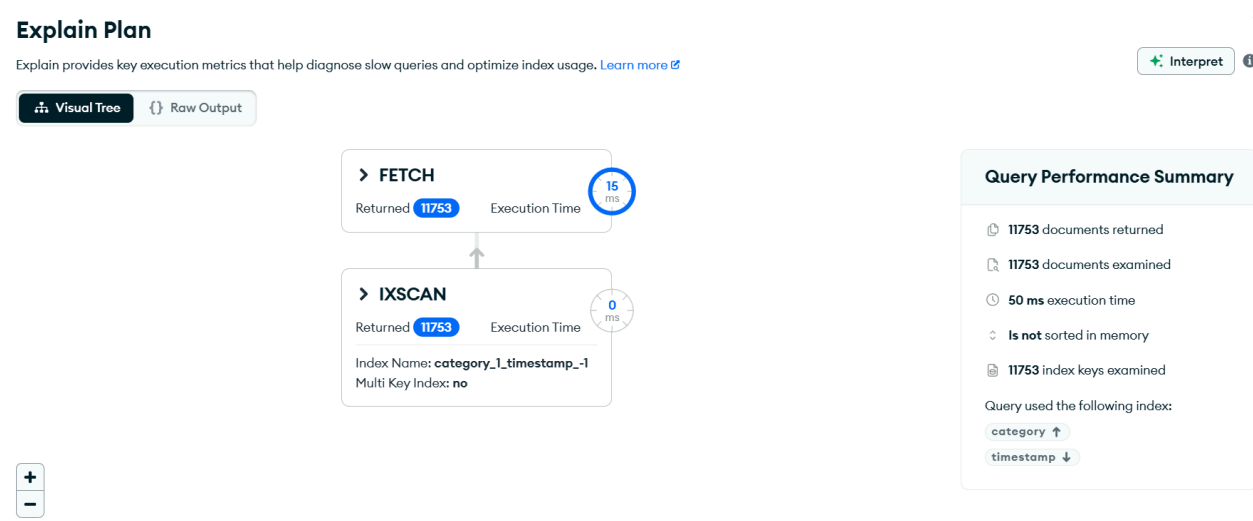
Програмний код скрипта create_index.py та підтвердження успішного створення індексу для поля category в терміналі

```
query_no_index.py > ...
1  from pymongo import MongoClient
2  import time
3
4  # Підключення до бази даних
5  client = MongoClient("mongodb://localhost:27017")
6  db = client["performance_test"]
7  collection = db["sales"]
8
9  print("Виконуємо пошук без індексу...")
10
11 # Фіксуємо час початку
12 start_time = time.time()
13
14 # Виконуємо пошук документів, де категорія "Electronics", і перетворюємо в список
15 results = list(collection.find({"category": "Electronics"}))
16
17 # Фіксуємо час завершення
18 end_time = time.time()
19
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

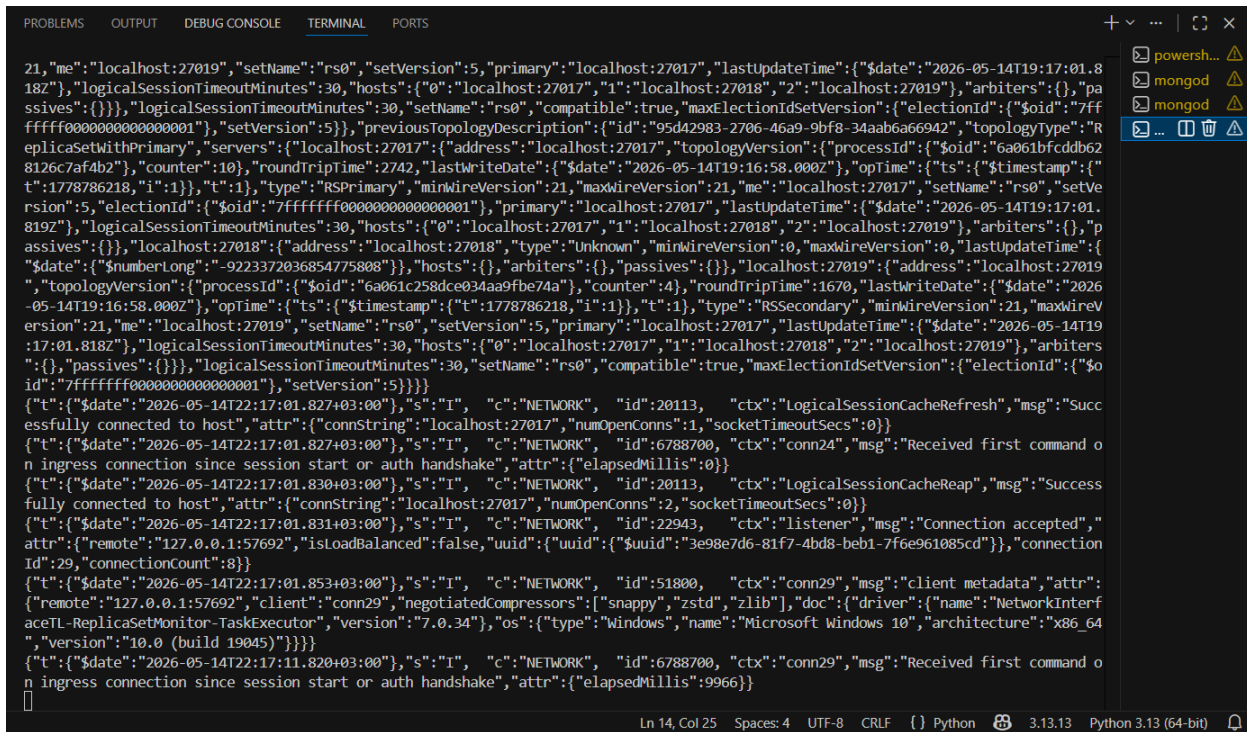
```
PS C:\MongoDB\Labs> python query_no_index.py
Виконуємо пошук без індексу...
Знайдено документів: 20100
Time taken: 0.124585 seconds
PS C:\MongoDB\Labs>
```

Результат виконання запиту після створення індексу (час пошуку значно скоротився завдяки використанню IXSCAN)



Аналіз плану виконання запиту (Explain Plan) після створення складеного індексу (category, timestamp). Візуалізація підтверджує використання складеного індексу (стадія IXSCAN), що забезпечує максимальну оптимізацію та високу швидкість вибірки даних.

1. Підготовка середовища та створення структури папок Для коректної роботи реплікаційного набору було створено окремі директорії для кожного вузла системи: data/rs0-1, data/rs0-2 та data/rs0-3. Це дозволяє кожному екземпляру MongoDB мати власний простір для зберігання даних та журналів логів.



```
21,"me":"localhost:27019","setName":"rs0","setVersion":5,"primary":"localhost:27017","lastUpdateTime":{"$date":"2026-05-14T19:17:01.818Z"},"logicalSessionTimeoutMinutes":30,"hosts":{"0":"localhost:27017","1":"localhost:27018","2":"localhost:27019"},"passives":{},"logicalSessionTimeoutMinutes":30,"setName":"rs0","compatible":true,"maxElectionIdSetVersion":{"electionId":{"$oid":"7ffff0000000000000000001"},"setVersion":5},"previousTopologyDescription":{"id":"95d42983-2706-46a9-9bf8-34aab6a66942","topologyType":"ReplicaSetWithPrimary","servers":{"localhost:27017":{"address":"localhost:27017","topologyVersion":{"processId":{"$oid":"6a061bfcddb628126c7af4b2"},"counter":10},"roundTripTime":2742,"lastWriteDate":{"$date":"2026-05-14T19:16:58.000Z"},"optime":{"ts":{"timestamp":{"t":"1778786218","i":1},"t":1},"type":"RSPrimary"},"minWireVersion":21,"maxWireVersion":21,"me":"localhost:27017","setName":"rs0","setVersion":5,"electionId":{"$oid":"7ffff0000000000000000001"},"primary":"localhost:27017","lastUpdateTime":{"$date":"2026-05-14T19:17:01.819Z"},"logicalSessionTimeoutMinutes":30,"hosts":{"0":"localhost:27017","1":"localhost:27018","2":"localhost:27019"},"arbiters":{"passives":{},"localhost:27018":{"address":"localhost:27018","type":"Unknown","minWireVersion":0,"maxWireVersion":0,"lastUpdateTime":{"$date":{"numberLong":"-9223372036854775808"},"hosts":{"arbiters":{"passives":{},"localhost:27019":{"address":"localhost:27019","topologyVersion":{"processId":{"$oid":"6a061c258dce034aa9fbe74a"},"counter":4},"roundTripTime":1670,"lastWriteDate":{"$date":"2026-05-14T19:16:58.000Z"},"optime":{"ts":{"timestamp":{"t":"1778786218","i":1},"t":1},"type":"RSSecondary"},"minWireVersion":21,"maxWireVersion":21,"me":"localhost:27019","setName":"rs0","setVersion":5,"primary":"localhost:27017","lastUpdateTime":{"$date":"2026-05-14T19:17:01.818Z"},"logicalSessionTimeoutMinutes":30,"hosts":{"0":"localhost:27017","1":"localhost:27018","2":"localhost:27019"},"arbiters":{"passives":{},"logicalSessionTimeoutMinutes":30,"setName":"rs0","compatible":true,"maxElectionIdSetVersion":{"electionId":{"$oid":"7ffff0000000000000000001"},"setVersion":5}}}}}}{"t":{"$date":"2026-05-14T22:17:01.827403:00"},"s":"I", "c":"NETWORK", "id":20113, "ctx":"LogicalSessionCacheRefresh","msg":"Successfully connected to host","attr":{"connString":"localhost:27017","numOpenConns":1,"socketTimeoutSecs":0}}{"t":{"$date":"2026-05-14T22:17:01.827403:00"},"s":"I", "c":"NETWORK", "id":6788700, "ctx":"conn24","msg":"Received first command on ingress connection since session start or auth handshake","attr":{"elapsedMillis":0}}{"t":{"$date":"2026-05-14T22:17:01.830403:00"},"s":"I", "c":"NETWORK", "id":20113, "ctx":"LogicalSessionCacheReap","msg":"Successfully connected to host","attr":{"connString":"localhost:27017","numOpenConns":2,"socketTimeoutSecs":0}}{"t":{"$date":"2026-05-14T22:17:01.831403:00"},"s":"I", "c":"NETWORK", "id":22943, "ctx":"listener","msg":"Connection accepted","attr":{"remote":"127.0.0.1:57692","isLoadBalanced":false,"uuid":{"$uuid":"3e98e7d6-81f7-4bd8-beb1-7f6e961085cd"},"connectionId":29,"connectionCount":8}}{"t":{"$date":"2026-05-14T22:17:01.853403:00"},"s":"I", "c":"NETWORK", "id":51800, "ctx":"conn29","msg":"client metadata","attr":{"remote":"127.0.0.1:57692","client":"conn29","negotiatedCompressors":["snappy","zstd","zlib"],"doc":{"driver":{"name":"NetworkInterfaceTL-ReplicaSetMonitor-TaskExecutor","version":"7.0.34"},"os":{"type":"windows","name":"Microsoft Windows 10","architecture":"x86_64","version":"10.0 (build 19045)"}}}}{"t":{"$date":"2026-05-14T22:17:11.820403:00"},"s":"I", "c":"NETWORK", "id":6788700, "ctx":"conn29","msg":"Received first command on ingress connection since session start or auth handshake","attr":{"elapsedMillis":9966}}
```

```
> rs.initiate()
< {
  info2: 'no configuration specified. Using a default configuration for the set',
  me: 'localhost:27017',
  ok: 1
}
> rs.add("localhost:27018")
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778786023, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778786023, i: 1 })
}
> rs.add("localhost:27019")
< {
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778786033, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778786033, i: 1 })
}
```

```

> rs.status()
< {
  set: 'rs0',
  date: 2026-05-14T19:14:05.465Z,
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    lastCommittedWallTime: 2026-05-14T19:13:57.178Z,
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    durableOpTime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    lastAppliedWallTime: 2026-05-14T19:13:57.178Z,
    lastDurableWallTime: 2026-05-14T19:13:57.178Z
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1778786027, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: 2026-05-14T19:12:47.973Z,
    electionTerm: Long('1'),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1778785967, i: 1 }), t: Long('-1') },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1778785967, i: 1 }), t: Long('-1') },
    numVotesNeeded: 1,
    priorityAtElection: 1,
    electionTimeoutMillis: Long('10000'),
    newTermStartDate: 2026-05-14T19:12:48.036Z,
    wMajorityWriteAvailabilityDate: 2026-05-14T19:12:48.070Z
  },
  members: [
    {
      _id: 0,
      name: 'localhost:27017',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 769,
      optime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
      optimeDate: 2026-05-14T19:13:57.000Z,
      lastAppliedWallTime: 2026-05-14T19:13:57.178Z,
      lastDurableWallTime: 2026-05-14T19:13:57.178Z,
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: 'Could not find member to sync from',
      electionTime: Timestamp({ t: 1778785967, i: 2 }),
      electionDate: 2026-05-14T19:12:47.000Z,
      configVersion: 5,

```

```

    _id: 0,
    name: 'localhost:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 769,
    optime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    optimeDate: 2026-05-14T19:13:57.000Z,
    lastAppliedWallTime: 2026-05-14T19:13:57.178Z,
    lastDurableWallTime: 2026-05-14T19:13:57.178Z,
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: 'Could not find member to sync from',
    electionTime: Timestamp({ t: 1778785967, i: 2 }),
    electionDate: 2026-05-14T19:12:47.000Z,
    configVersion: 5,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 1,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 21,
    optime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    optimeDate: 2026-05-14T19:13:57.000Z,
    optimeDurableDate: 2026-05-14T19:13:57.000Z,
    lastAppliedWallTime: 2026-05-14T19:13:57.178Z,
    lastDurableWallTime: 2026-05-14T19:13:57.178Z,
    lastHeartbeat: 2026-05-14T19:14:05.201Z,
    lastHeartbeatRecv: 2026-05-14T19:14:05.205Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 5,
    configTerm: 1
  },
  {
    _id: 2,
    name: 'localhost:27019',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 12,
    optime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
    optimeDurable: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },

```

```

    },
    {
      _id: 2,
      name: 'localhost:27019',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 12,
      optime: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
      optimeDurable: { ts: Timestamp({ t: 1778786037, i: 1 }), t: Long('1') },
      optimeDate: 2026-05-14T19:13:57.000Z,
      optimeDurableDate: 2026-05-14T19:13:57.000Z,
      lastAppliedWallTime: 2026-05-14T19:13:57.178Z,
      lastDurableWallTime: 2026-05-14T19:13:57.178Z,
      lastHeartbeat: 2026-05-14T19:14:05.199Z,
      lastHeartbeatRecv: 2026-05-14T19:14:03.699Z,
      pingMs: Long('0'),
      lastHeartbeatMessage: '',
      syncSourceHost: 'localhost:27017',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 5,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778786037, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778786037, i: 1 })
}
rs0 [direct: primary] test>

```

Перевірка стану кластера командою `rs.status()`. Результат підтверджує, що реплікаційний набір `rs0` успішно ініціалізовано: один вузол отримав статус PRIMARY, а два інші — SECONDARY.

```

}
> db.adminCommand({ shutdown: 1 })
MongoNetworkError: read ECONNRESET
test>

```

```

members: [
  {
    _id: 0,
    name: 'localhost:27017',
    health: 0,
    state: 0,
    stateStr: '(not reachable/healthy)',
    uptime: 0,
    optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDate: 2026-05-14T08:00:00.000Z,
    optimeDurableDate: 2026-05-14T08:00:00.000Z,
    lastAppliedWallTime: 2026-05-14T19:19:38.567Z,
    lastDurableWallTime: 2026-05-14T19:19:38.567Z,
    lastHeartbeat: 2026-05-14T19:22:13.157Z,
    lastHeartbeatRecv: 2026-05-14T19:19:43.120Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: 'Error connecting to localhost:27017 (127.0.0.1:27017) : caused by :: onInvoke :: caused by :: No connection could be made because the target machine actively refused it.',
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    configVersion: 5,
    configTerm: 2
  },
  {
    _id: 1,
    name: 'localhost:27018',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 1232,
    optime: { ts: Timestamp({ t: 1778786528, i: 1 }), t: Long('2') },
    optimeDate: 2026-05-14T19:22:08.000Z,
    lastAppliedWallTime: 2026-05-14T19:22:08.582Z,
    lastDurableWallTime: 2026-05-14T19:22:08.582Z,
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1778786368, i: 2 }),
    electionDate: 2026-05-14T19:19:18.000Z,
    configVersion: 5,
    configTerm: 2,
    self: true,
    lastHeartbeatMessage: ''
  },
]

```

```

    _id: 2,
    name: 'localhost:27019',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 501,
    optime: { ts: Timestamp({ t: 1778786528, i: 1 }), t: Long('2') },
    optimeDurable: { ts: Timestamp({ t: 1778786528, i: 1 }), t: Long('2') },
    optimeDate: 2026-05-14T19:22:08.000Z,
    optimeDurableDate: 2026-05-14T19:22:08.000Z,
    lastAppliedWallTime: 2026-05-14T19:22:08.582Z,
    lastDurableWallTime: 2026-05-14T19:22:08.582Z,
    lastHeartbeat: 2026-05-14T19:22:12.720Z,
    lastHeartbeatRecv: 2026-05-14T19:22:13.190Z,
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27018',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 5,
    configTerm: 2
  }
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1778786528, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1778786528, i: 1 })
}

```

6. Теоретичні аспекти горизонтального масштабування (Шардинг)

Механізм шардингу в MongoDB призначений для забезпечення горизонтального масштабування шляхом розподілу великих масивів даних між декількома фізичними серверами (шардами). Це дозволяє системі опрацьовувати обсяги інформації та запитів, що перевищують апаратні можливості одного окремого вузла.

Основними компонентами такої архітектури є:

- **Shard (шард)** — окремий вузол або реплікаційний набір, що зберігає частину загальних даних;
- **Mongos** — маршрутизатор, який спрямовує запити клієнтів до відповідних шардів;
- **Config Servers** — сервери конфігурації, що зберігають метадані про розподіл даних у кластері.

У межах даної практичної роботи повноцінне налаштування кластера з шардингом не було реалізовано через технічні обмеження хмарної платформи (MongoDB Atlas Free Tier) та локального середовища, які не надають доступу до керування компонентами маршрутизації та конфігураційними серверами у безкоштовній версії. Проте, принципи функціонування даної технології, алгоритми розподілу даних (Hashed vs Ranged sharding) та роль ключа шардування були вивчені теоретично.

Перевірка відмовостійкості кластера. Після примусового вимкнення головного вузла (порт 27017), кластер автоматично провів вибори та призначив новий вузол (27018) як PRIMARY, забезпечивши безперебійну роботу

Висновок

Під час виконання лабораторної роботи було досліджено механізми оптимізації та забезпечення високої доступності в NoSQL базі даних MongoDB.

Індексація: На практиці підтверджено, що створення індексів значно прискорює пошукові запити, зменшуючи кількість сканованих документів (COLLSCAN змінюється на IXSCAN), що є критичним для продуктивності великих систем.

Реплікація: Було успішно розгорнуто реплікаційний набір (Replica Set) з трьох вузлів. Це дозволило реалізувати механізм автоматичного відновлення після збоїв (failover).

Експеримент із вимкненням головного вузла продемонстрував здатність MongoDB самостійно обирати нового лідера та продовжувати роботу без втручання адміністратора.

Отримані навички є фундаментальними для побудови сучасних розподілених систем, де швидкість доступу до даних та їх збереження у разі технічних несправностей є ключовими вимогами.

